業務工数システム 引き継ぎ教育資料

このフォルダ（`kosu/`）には、本Webアプリ「業務工数システム」を **作成・運用・保守** していくために必要な知識を、**プログラミング完全初心者** でも学べるようにまとめた教育資料が入っています。

> 🔰 **このシステムを引き継いだ方は、最初に「000_引き継ぎ書_最初に読む」を読んでください。** 受け取る物のチェックリスト、最初の 1 か月にやること、変更が本番に出る仕組み（`master` に push すると、そのまま本番へ出ます）、画面とプログラムの対応表、押す前にバックアップが要る操作、前任者に確かめることを、1 冊にまとめてあります。
>
> **アプリ本体と、この資料の置き場所:** アプリのプログラム（ソースコード）は、別の場所の `kosu_react` フォルダの中の `hozen-kosu-React/` にあります（以下、資料の中で「プロジェクト直下」と書いたら、この `hozen-kosu-React/` のことです）。この資料は、それを説明する読み物で、アプリの動きには関係しません。資料を書き換えても、アプリは変わりません。

---

0. この資料の対象読者

- これまでプログラミングをほとんど（あるいは全く）やったことがない方
- このアプリの保守・運用を引き継ぐことになった方
- 「何から手をつければいいか分からない」という状態の方

専門用語には初出のときに必ず説明を入れています。サンプルコードは原則 **1行ずつコメント** を付けています。分からない単語が出てきたら、まず巻末の「10_用語集」を引いてください。

---

1. このアプリは何をするもの？（30秒で理解）

工場などの現場で、従業員が「**いつ・どの作業に・何時間使ったか**（＝**工数**）」を記録・集計するためのWebアプリです。

- 従業員が自分の作業時間を入力する
- 休憩時間を差し引いて正味の工数を自動計算する
- 班（チーム）単位・ショップ（職場）単位で集計・グラフ表示する
- 管理者がデータのバックアップ・復元、操作履歴の確認を行う

詳しくは「00_導入と全体像」で説明します。

---

2. 技術構成（ざっくり）

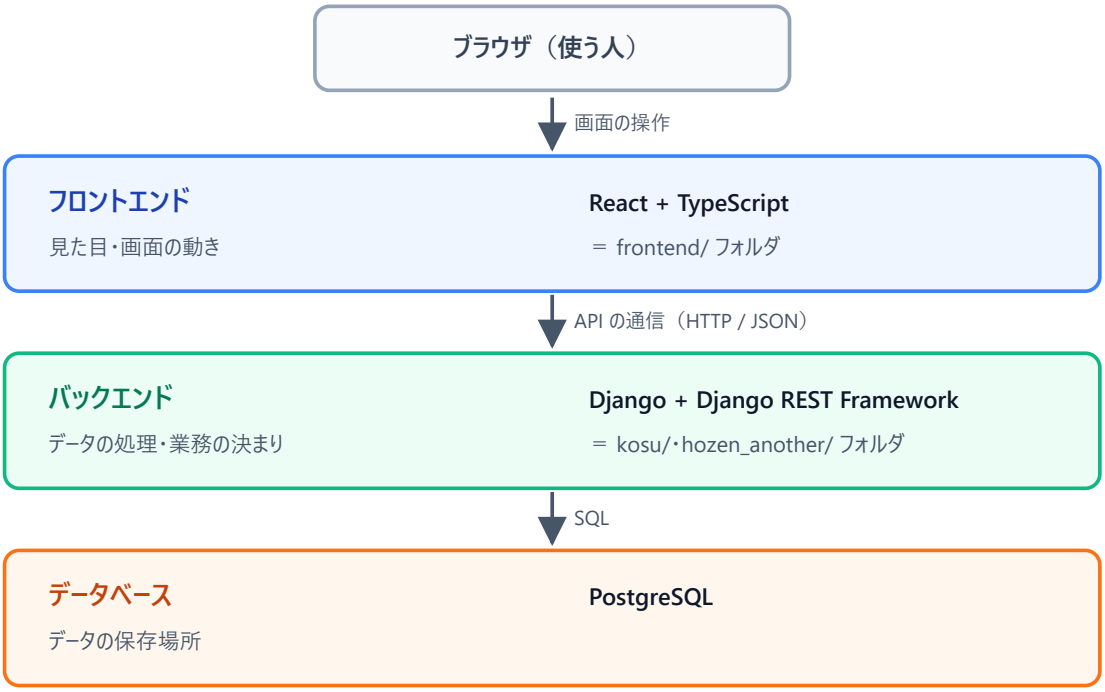


図: 技術の構成。ブラウザで動く画面（React）が、API でサーバー（Django）に頼み、Django がデータベース（PostgreSQL）を読み書きする

「フロントエンド」「バックエンド」「API」などの言葉が分からなくても大丈夫です。後の資料で順を追って説明します。

---

3. 資料の一覧と読む順番

上から順番に読む ことを強くおすすめします。前の資料の知識を前提に書いています。

No.	ファイル	内容	重要度
000	000_引き継ぎ書_最初に読む.md	**引き継いだ人が最初に読む巻**。全体図・受け取る物・最初の 1 か月・本番に出る仕組み・全 49 画面とプログラムの対応表・危ない操作・よくある依頼・前任者に確かめること	★★★
00a	00a_はじめての方へ_超入門.md	**完全初心者の最初の一歩**。ファイル/フォルダ/パス/ターミナル/コマンド/ブラウザの超基礎	★★★
00	00_導入と全体像.md	アプリの目的・全体構造・**なぜこの技術を選んだか**（技術選定理由）	★★★
01	01_環境構築.md	VSCode・Git・Node.js・Python・PostgreSQLの導入、GitHubからのクローン、起動手順	★★★
02	02_Git_GitHub.md	バージョン管理の基礎、毎日使うGitコマンド、GitHub運用	★★★
03	03_Web基礎_HTML_CSS.md	Webページの仕組み、HTML・CSSの基礎、本アプリでの使われ方	★★
04	04_JavaScript_TypeScript.md	JavaScript・TypeScriptの基礎、本アプリで多用する書き方	★★★
05	05_React_フロントエンド.md	Reactの考え方、本アプリのフロント構造（ルーティング・API通信・画面）	★★★
06	06_Python基礎.md	Pythonの基礎、本アプリのコードを読むための知識	★★★
07	07_Django_バックエンド.md	Django・DRFの仕組み、本アプリのAPI・非同期処理・履歴記録	★★★
08	08_データベースとモデル.md	データベース設計、各テーブル（モデル）の意味、業務ルール	★★★
09	09_運用_保守_デプロイ.md	日々の運用、バックアップ・復元、Azureへのデプロイ、障害対応	★★★
10	10_用語集.md	この資料に出てくる用語の早見表	★★
11	11_使用技術ライブラリ全カタログ.md	使用している**全ライブラリ**の解説。未使用（レガシー）候補も正直に明記	★★
12	12_テストの書き方と実行.md	フロント(Vitest)・バック(Django)の自動テストの読み方・書き方・実行	★★
13	13_付録_出てくるコードとコマンドの一覧.md	00〜12 と 15 に出てくるコードの部品・コマンドを**機能ごと**に並べ、意味と初めて出てくる所を付けた一覧（「これ何だっけ？」をコードから引く）	★★
14	14_付録_逆引き_やりたいことからコードを探す.md	「一覧をしぼり込みたい」「Excel に書き出したい」のような**やりたいことから書き方を引く**逆引き。本文に無い、合わせて覚えるとよいコードも	★★

No.	ファイル	内容	重要度
15	15_直すべき所と直し方.md	アプリの**直すべき所を全部**（安全の問題・不具合・整理）、場所（ファイルと行）・何が困るか・直した後のコード・確かめ方つきで並べた改修の一覧。直す順番も	★★★

---

4. 学習ロードマップ（目安）

完全初心者を想定した、無理のない進め方の目安です。**焦らず、手を動かしながら** 進めてください。

フェーズ1：まず動かす（1〜2週間）

0. まず `000`（引き継ぎ書）を読み、受け取る物をそろえる。パソコン操作に不安があれば、続けて `00a`（超入門）を読む
1. `00` を読んで全体像をつかむ（理解できなくてもOK、雰囲気をつかむ）
2. `01` を読みながら、自分のPCに開発環境を作る
3. `02` を読みながら、GitHubからコードを取得（クローン）できるようにする
4. アプリをローカル（自分のPC）で起動して、画面を触ってみる

> ✅ ゴール：自分のPCで `npm start` と `python manage.py runserver` を実行し、ブラウザでアプリが表示できる。

フェーズ2：言語の基礎（2〜4週間）

5. `03`（HTML/CSS）→ `04`（JS/TS）→ `06`（Python）を読む
6. 実際のコードを開いて「これはあの文法だな」と照らし合わせる

> ✅ ゴール：本アプリのコードを開いたとき、「何が書いてあるか雰囲気が読める」。

フェーズ3：フレームワークと本アプリの構造（3〜6週間）

7. `05`（React）と `07`（Django）を読む
8. `08`（データベース・モデル）でデータ構造を理解する
9. 小さな修正（文言の変更など）を試してみる

> ✅ ゴール：「この画面のこの文字を変えたい」を自力で実現できる。

フェーズ4：運用・保守(継続)

10. `09` を読み、バックアップ・復元・デプロイを実際に練習する
11. `15` の「🔴 すぐ」の物から、作業用のブランチで 1 つずつ直す

> ✅ ゴール：本番環境を壊さずに、運用作業とリリースができる。

---

5. この資料の使い方のコツ

- **分からない単語はすぐ調べる**：まず `10_用語集` 、なければ検索（「○○ とは」で調べると初心者向け解説が出ます）。
- **コードは写経する**：読むだけでなく、自分で打ち込むと身につきます。
- **壊しても大丈夫な環境で練習する**：ローカル環境やテスト用データで試し、いきなり本番を触らない（理由は `09` で説明）。
- **エラーは敵ではない**：エラーメッセージは「どこが悪いか」を教えてくれる味方です。メッセージをそのまま検索しましょう。

---

6. PDF版・Web版と、コードのコピーのしかた

各 `.md` （マークダウン）ファイルは、次の 2 つの形でも読めます。

形	置き場所	向いている使い方
PDF版	`kosu/pdf/`	VS Code・Adobe Acrobat・ブラウザーで読む／紙に印刷する
Web版（HTML版）	`kosu/html/` （入口は `index.html` ）。インターネット上の `https://oltotlo79-rgb.github.io/education/kosu/html/` にも同じ物があります	ブラウザーで読む。コードの「コピー」ボタンで、字下げもそのままコピーできる

コードのコピーのしかた（見ている物ごとに違います）：

- **PDF を VS Code で見ている時:** コードの枠の 1 行目の最初の文字の上から、最後の行の最後の文字の上までをマウスでなぞって選び、`Ctrl` ＋ `C` を押します。字下げ（行の先頭の空白）もそのままコピーできます（枠の上にオレンジの字で「なぞってコピーすると…」と書いてある枠だけは、次の「コピー」を使います）。
- **PDF を Adobe Acrobat（Acrobat Reader）で見ている時:** 枠の右上の青い「コピー」を押します。Acrobat の中に窓が開き、コードが全部青く選ばれているので、そのまま `Ctrl` ＋ `C` を押して、窓は「閉じる」で閉じます（窓の下の赤い字「警告：JavaScript ウィンドウ」は、PDF の中のボタンから開いた窓だという印で、エラーではありません）。Acrobat では、なぞってコピーしません（字下げと空の行が消えます）。

- **PDF を Chrome や Edge で見ている時:** 枠の右上の青い「コピー」を押すと、Web 版のその場所が開きます。そこで「コピー」ボタンを押します。Chrome や Edge の PDF では、なぞってコピーしないでください（字下げが縮まり、空の行が消えます。Python は字下げが崩れると動きません）。
- **Web 版を見ている時:** 枠の右上の「コピー」ボタンを押すだけです。

> **初めて出てきた言葉・書き方の説明（「初出の解説」）:** コードの枠の下の、色の付いた箱は、その巻で初めて出てきた関数・書き方・記号などを、短く説明した物です。あとで探したい時は、巻末の「13_付録_出てくるコードとコマンドの一覧」（コードから意味を引く）と「14_付録_逆引き_やりたいことからコードを探す」（やりたいことから書き方を引く）を使います。

PDF・Web版を作り直す手順は「09_運用_保守_デプロイ.md」の §17.2 にあります。

---

7. 困ったときの連絡先・情報源

- 「000_引き継ぎ書_最初に読む」の §9（よくある依頼）と §10（前任者に確かめること）を見ます。
- アプリ内の「問い合わせ」機能（`InquirPage`）で、過去の質問・回答が確認できます。
- コードの全体方針は、プロジェクト直下の `CLAUDE.md` にも記載があります。
- それでも解決しない場合は、前任者・社内のIT担当者に相談してください。

それでは、「00_導入と全体像.md」から始めましょう。